

Task Brief for Hard Benchmark Closure

For the teammate handling error analysis, feature importance, and hard-level result cleanup.

Send this PDF to ChatGPT

Use with Codex

RAG_Medical_Reranker

0. Context for ChatGPT / Codex

This document describes the current state of the **RAG_Medical_Reranker** project and the exact tasks needed to complete the hard benchmark analysis. If you give this document to ChatGPT, it should be able to guide you through the next coding and evaluation steps.

Repo: https://github.com/garrylee4mailbox-arch/RAG_Medical_Reranker

Main hard benchmark file: data/processed/candidates_hard.csv

Main hard result folder: results_hard/

1. Current Hard Benchmark Status

Item	Status
Hard candidates	Generated: data/processed/candidates_hard.csv
Scale	500 qids, 5,000 query-candidate pairs
Per qid design	1 positive + 7 embedding_hard_negative + 2 random_negative
Embedding baseline	Completed
Department bonus	Completed
Lightweight ML	Completed
BCE	Completed and pushed
BGE	Pending, handled by another teammate

2. Current Hard Benchmark Results

Method	P@1	P@3	P@5	MRR	NDCG@5	Latency
embedding_baseline	0.886	0.968	0.986	0.9285	0.9416	0.0000
embedding_department_bonus	0.886	0.970	0.986	0.9290	0.9420	0.0000
lightweight_ml	0.998	1.000	1.000	0.9990	0.9993	0.0004
BCE	0.884	0.988	0.998	0.9358	0.9516	0.5252
BGE	TBD	TBD	TBD	TBD	TBD	TBD

3. Your Main Mission

Your task is to complete the scientific analysis around the hard benchmark before the BGE result is available. You are not expected to rerun BCE or generate new candidates.

1. Add error analysis for hard benchmark ranking failures.
2. Add feature importance output for the lightweight ML logistic regression model.
3. Run both analyses and save outputs under `results_hard/`.
4. Do not overwrite controlled benchmark files under `results/`.
5. Do not modify `data/processed/candidates.csv`.
6. Do not run BGE unless explicitly assigned.

4. First Step: Sync the Repo

```
git clone https://github.com/garrylee4mailbox-arch/RAG_Medical_Reranker.git
cd RAG_Medical_Reranker
git pull origin main
```

If you already have the repo:

```
cd RAG_Medical_Reranker
git status --short
git pull origin main
```

Before making changes, confirm the hard files exist:

```
ls data/processed/candidates_hard.csv
ls results_hard/bce_scores.csv
ls results_hard/summary_metrics.csv
```

5. Task A - Add Error Analysis

Goal

Create a script that identifies where different rerankers succeed or fail. This is needed because the lightweight ML model performs extremely well, and we need to explain whether it is correcting meaningful hard retrieval mistakes.

New files

```
src/analysis/__init__.py
src/analysis/error_analysis.py
```

Required outputs

```
results_hard/error_cases.csv
results_hard/error_summary.csv
```

Case types to identify

- `embedding_wrong_ml_correct` : embedding_baseline positive rank > 1 and lightweight_ml positive rank = 1
- `bce_wrong_ml_correct` : bce positive rank > 1 and lightweight_ml positive rank = 1
- `embedding_correct_bce_wrong` : embedding_baseline positive rank = 1 and bce positive rank > 1
- `bce_correct_embedding_wrong` : bce positive rank = 1 and embedding_baseline positive rank > 1
- `all_available_wrong` : all available methods rank the positive below 1

6. Task B - Add Feature Importance

Goal

Update `src/ml/train_lightweight_ml.py` minimally so that it can export logistic regression coefficients. This will help explain why the feature-based ML reranker performs so strongly.

New optional argument

```
--feature-importance-out results_hard/ml_feature_importance.csv
```

Required output columns

```
feature, coefficient, abs_coefficient
```

The output should be sorted by `abs_coefficient` descending.

7. Codex Prompt to Use

Copy the following prompt into Codex. It is written to be self-contained.

```
We are working on the GitHub repo RAG_Medical_Reranker.

Current project state:
- Hard benchmark file: data/processed/candidates_hard.csv
- Hard result folder: results_hard/
- Hard benchmark scale: 500 qids, 5000 rows
- Each qid has 1 positive, 7 embedding_hard_negative, and 2 random_negative.

Current hard benchmark results:
- embedding_baseline: P@1 = 0.886, MRR = 0.9285, NDCG@5 = 0.9416
- embedding_department_bonus: P@1 = 0.886, MRR = 0.9290, NDCG@5 = 0.9420
- lightweight_ml: P@1 = 0.998, MRR = 0.9990, NDCG@5 = 0.9993
- bce: P@1 = 0.884, MRR = 0.9358, NDCG@5 = 0.9516
- bge is still pending.
```

Task 1: Create error analysis.

Create:

- src/analysis/__init__.py
- src/analysis/error_analysis.py

The script should read:

```
--candidates data/processed/candidates_hard.csv
--score-files method=path pairs, defaulting to:
  embedding_baseline=results_hard/embedding_baseline_scores.csv
  embedding_department_bonus=results_hard/embedding_department_bonus_scores.csv
  lightweight_ml=results_hard/ml_scores.csv
  bce=results_hard/bce_scores.csv
  bge=results_hard/bge_scores.csv
```

It should write:

```
--out-cases results_hard/error_cases.csv
--out-summary results_hard/error_summary.csv
```

Requirements:

1. Use `src.utils.validate_candidates` and `validate_scores` if available.
2. Skip missing score files gracefully with a warning.
3. Merge scores with candidates by `qid` and `context_id`.
4. For each `qid` and method, find the positive candidate rank and the top-1 candidate.
5. Identify case types:
 - `embedding_wrong_ml_correct`
 - `bce_wrong_ml_correct`
 - `embedding_correct_bce_wrong`
 - `bce_correct_embedding_wrong`
 - `all_available_wrong`
6. The cases CSV should include `qid`, `case_type`, `question`, `positive_context_id`, `positive_context`, positive ranks for each method, top1 `context_id` for each method, top1 `source_type` for key methods, and top1 context text for key methods.
7. Add `--max-context-chars` default 500 to truncate long context text.
8. The summary CSV should include counts and percentages by `case_type` plus method-level rank distribution if convenient.
9. Do not modify candidates files.
10. Do not overwrite existing score files.

Task 2: Add feature importance output to `src/ml/train_lightweight_ml.py`.

Add optional CLI argument:

```
--feature-importance-out
```

When `--model logreg` and `--feature-importance-out` is provided:

1. Extract LogisticRegression coefficients.
2. Pair them with the feature names actually used for training.
3. Write CSV columns:
 - feature, coefficient, abs_coefficient
4. Sort by `abs_coefficient` descending.
5. Use `ensure_parent` before writing.

Do not change existing behavior unless this new argument is used.

Do not change the current score output format.

Do not use `label`, `source_type`, `qid`, or `context_id` as features.

Verification commands:

```
python -m compileall src/analysis/error_analysis.py src/ml/train_lightweight_ml.py
```

```
python -m src.ml.train_lightweight_ml --candidates data/processed/candidates_hard.csv --out results_hard/ml_scores.csv --metrics-out results_hard/ml_classification_metrics.csv --model logreg --score-all --feature-importance-out results_hard/ml_feature_importance.csv
```

```
python -m src.analysis.error_analysis --candidates data/processed/candidates_hard.csv --out-cases results_hard/error_cases.csv --out-summary results_hard/error_summary.csv --score-files embedding_baseline=results_hard/embedding_baseline_scores.csv embedding_department_bonus=results_hard/embedding_department_bonus_scores.csv lightweight_ml=results_hard/ml_scores.csv bce=results_hard/bce_scores.csv bge=results_hard/bge_scores.csv
```

After running, print:

- first 10 rows of results_hard/ml_feature_importance.csv
- full results_hard/error_summary.csv
- first 5 rows of results_hard/error_cases.csv

8. Commands After Codex Implementation

Run compile check:

```
python -m compileall src/analysis/error_analysis.py src/ml/train_lightweight_ml.py
```

Run feature importance:

```
python -m src.ml.train_lightweight_ml `
--candidates data/processed/candidates_hard.csv `
--out results_hard/ml_scores.csv `
--metrics-out results_hard/ml_classification_metrics.csv `
--model logreg `
--score-all `
--feature-importance-out results_hard/ml_feature_importance.csv
```

Run error analysis:

```
python -m src.analysis.error_analysis `
--candidates data/processed/candidates_hard.csv `
--out-cases results_hard/error_cases.csv `
--out-summary results_hard/error_summary.csv `
--score-files `
embedding_baseline=results_hard/embedding_baseline_scores.csv `
embedding_department_bonus=results_hard/embedding_department_bonus_scores.csv `
lightweight_ml=results_hard/ml_scores.csv `
bce=results_hard/bce_scores.csv `
bge=results_hard/bge_scores.csv
```

9. Expected Files to Commit

After successful implementation and execution, commit only relevant files:

```
git status --short

git add src/analysis/__init__.py `
src/analysis/error_analysis.py `
src/ml/train_lightweight_ml.py `
results_hard/ml_feature_importance.csv `
results_hard/error_cases.csv `
results_hard/error_summary.csv

git commit -m "Add hard benchmark error analysis and ML interpretability"
git push origin main
```

Do not use git add . There may be unrelated report PDFs, slides, temporary files, or local experiments.

10. How to Write the Analysis in the Report

After error analysis and feature importance are complete, the report should answer two questions:

1. **What mistakes does lightweight ML fix?** Use `error_cases.csv` to show examples where embedding or BCE fails but ML succeeds.
2. **Why does lightweight ML work so well?** Use `ml_feature_importance.csv` to show whether the model mainly depends on `embedding_score` or also uses keyword overlap, department match, and length features.

Suggested wording:

The lightweight ML reranker should be interpreted as a supervised feature-based reranker. Its strong performance does not mean that logistic regression has deeper semantic understanding than BCE. Instead, it shows that, under a fixed retrieval-hard RAG distribution, supervised calibration of retrieval signals can be extremely effective and low-latency.

11. Coordination with BGE

BGE is still pending. Your task does not depend on BGE. The error analysis script must skip missing `results_hard/bge_scores.csv` gracefully. Once BGE is available, rerun:

```
python -m src.evaluate --candidates data/processed/candidates_hard.csv --out results_hard/
summary_metrics.csv --score-files embedding_baseline=results_hard/embedding_baseline_scores.csv
embedding_department_bonus=results_hard/embedding_department_bonus_scores.csv lightweight_ml=results_hard/
ml_scores.csv bce=results_hard/bce_scores.csv bge=results_hard/bge_scores.csv

python -m src.analysis.error_analysis --candidates data/processed/candidates_hard.csv --out-cases
results_hard/error_cases.csv --out-summary results_hard/error_summary.csv --score-files
embedding_baseline=results_hard/embedding_baseline_scores.csv embedding_department_bonus=results_hard/
embedding_department_bonus_scores.csv lightweight_ml=results_hard/ml_scores.csv bce=results_hard/
bce_scores.csv bge=results_hard/bge_scores.csv
```